Optimisation de l'utilisation des clusters et des GPU avec l'exécution d'IA

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/fr-fr/netapp-solutions/ai/osrunai_run_ai_installation.html on March 19, 2024. Always check docs.netapp.com for the latest.

Sommaire

Optimisation de l’utilisation des clusters et des GPU avec Run:ai . . . 1
Exécutez : installation ai . . . 1
Exécution :tableaux de bord et vues d’IA . . . 1
Création de projets pour les équipes de data science et allocation de GPU . . . 2
Soumission des travaux dans l’interface de ligne de commande Run:ai . . . 3
Optimiser l’utilisation des clusters . . . 6
Allocation fractionnelle de GPU pour des charges de travail moins exigeantes ou interactives . . . 7
Une utilisation élevée des clusters grâce à une allocation GPU sur-quota . . . 8
Équité de l’allocation des ressources de base . . . 10
Équité excessive . . . 11
Sauvegarde des données vers un volume persistant provisionné par Trident . . . 13

Optimisation de l'utilisation des clusters et des GPU avec Run:ai

Les sections suivantes décrivent l'installation de Run:ai, les scénarios de test et les résultats obtenus lors de cette validation.

Le fonctionnement et les performances de ce système ont été validées à l'aide d'outils de test standard du secteur, dont des bancs d'essai TensorFlow. Le dataset ImageNet a été utilisé pour entraîner le modèle ResNet-50, qui est un modèle d'apprentissage profond connu des réseaux neuronaux convolutifs (CNN) pour la classification d'images. RESNET-50 offre des résultats d'entraînement précis avec un temps de traitement plus rapide, ce qui nous a permis d'obtenir une demande suffisante en matière de stockage.

Exécutez : installation ai

Pour installer Run:ai, procédez comme suit :

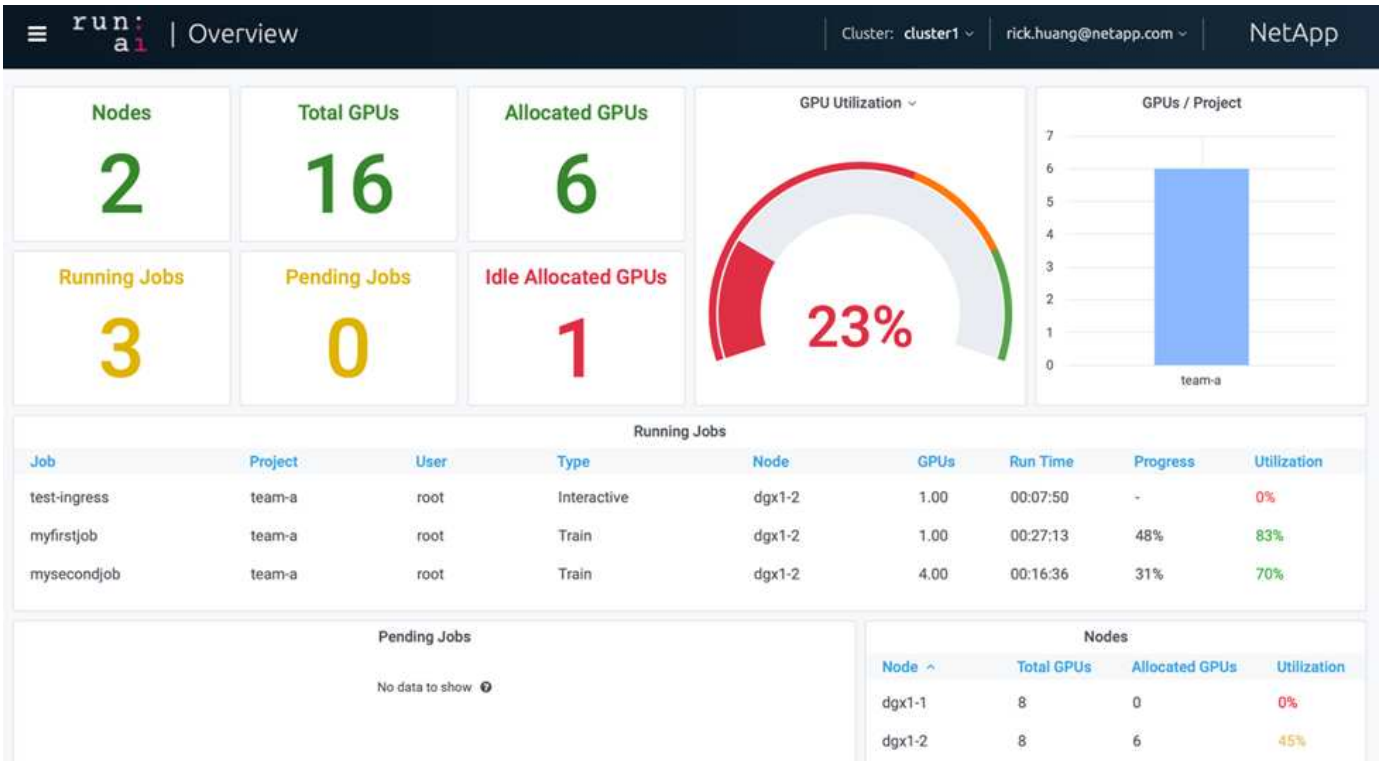
1. Installez le cluster Kubernetes à l'aide de DeepOps et configurez la classe de stockage par défaut de NetApp.
2. Préparez les nœuds GPU :
   a. Vérifiez que les pilotes NVIDIA sont installés sur les nœuds GPU.
   b. Vérifiez-le `nvidia-docker` est installé et configuré comme exécution docker par défaut.
3. Installer Run:ai :
   a. Connectez-vous au "Exécution : interface d'administration d'IA" pour créer le cluster.
   b. Téléchargez le créé `runai-operator-<clustername>.yaml` fichier.
   c. Appliquer la configuration de l'opérateur au cluster Kubernetes.

   ```
   kubectl apply -f runai-operator-<clustername>.yaml
   ```

4. Vérifiez l'installation :
   a. Accédez à "https://app.run.ai/".
   b. Accédez au tableau de bord Présentation.
   c. Vérifiez que le nombre de GPU situé en haut à droite indique le nombre attendu de GPU et de nœuds GPU dans la liste des serveurs.pour plus d'informations sur le déploiement Run:ai, consultez "Installation de Run:ai sur un cluster Kubernetes sur site" et "Installation de la CLI Run:ai".

Exécution :tableaux de bord et vues d'IA

Une fois l'installation de Run:ai sur votre cluster Kubernetes et la configuration correcte des conteneurs, vous voyez les tableaux de bord et les vues suivants sur "https://app.run.ai" dans votre navigateur, comme illustré dans la figure suivante.



Le cluster fournit au total 16 GPU par deux nœuds DGX-1. Vous pouvez voir le nombre de nœuds, le nombre total de GPU disponibles, les GPU alloués avec des charges de travail, le nombre total de tâches en cours d'exécution, les tâches en attente et les GPU alloués inactifs. Sur le côté droit, le diagramme à barres affiche les GPU par projet, qui récapitule la manière dont les différentes équipes utilisent les ressources du cluster. Au milieu se trouve la liste des travaux en cours d'exécution avec des détails sur le travail, y compris le nom du travail, le projet, l'utilisateur, le type de travail, Le nœud sur lequel chaque travail est exécuté, le nombre de GPU alloués pour ce travail, la durée d'exécution actuelle du travail, la progression du travail en pourcentage et l'utilisation du GPU pour ce travail. Notez que le cluster est sous-utilisé (taux d'utilisation des GPU à 23 %) car seules trois tâches en cours d'exécution sont soumises par une seule équipe (`team-a`).

Dans la section suivante, nous allons voir comment créer plusieurs équipes dans l'onglet projets et allouer des GPU à chaque équipe pour optimiser l'utilisation du cluster et gérer les ressources lorsqu'il y a de nombreux utilisateurs par cluster. Les scénarios de test reproduisent les environnements d'entreprise dans lesquels la mémoire et les ressources GPU sont partagées entre les charges de travail d'entraînement, d'inférence et interactives.

Création de projets pour les équipes de data science et allocation de GPU

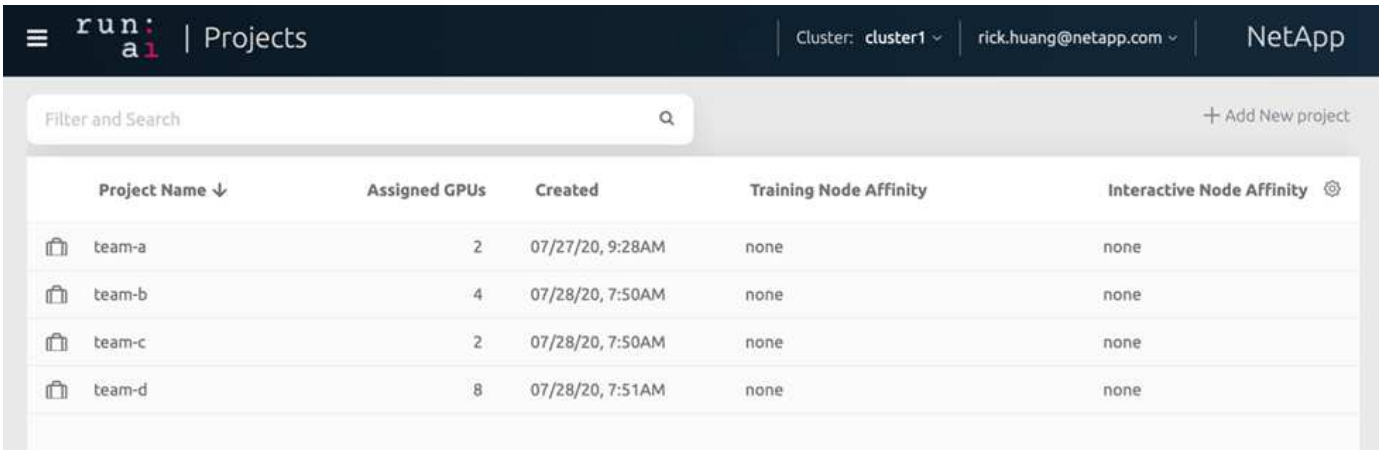
Les chercheurs peuvent envoyer des workloads via les processus Run:ai CLI, Kubeflow ou similaires. Afin de rationaliser l'allocation des ressources et de créer des priorités, Run:ai introduit le concept de projets. Les projets sont des entités de quotas qui associent un nom de projet à l'allocation et aux préférences GPU. Cette méthode simple et pratique vous permet de gérer plusieurs équipes de data science.

Un chercheur soumettant une charge de travail doit associer un projet à une demande de charge de travail. Le planificateur Run:ai compare la demande aux allocations en cours et au projet et détermine si la charge de travail peut être allouée à des ressources ou si elle doit rester dans un état en attente.

En tant qu'administrateur système, vous pouvez définir les paramètres suivants dans l'onglet exécution:projets ai :

- **Modèles de projets.** définissez un projet par utilisateur, définissez un projet par équipe d'utilisateurs et définissez un projet par projet d'organisation réel.
- **Quotas de projet.** chaque projet est associé à un quota de GPU qui peut être alloué pour ce projet en même temps. C'est un quota garanti. Les chercheurs qui utilisent ce projet sont garantis d'obtenir ce nombre de GPU, quel que soit l'état du cluster. En règle générale, la somme de l'allocation du projet doit être égale au nombre de GPU dans le cluster. Au-delà de cela, un utilisateur de ce projet peut recevoir un surquota. Tant que les GPU ne sont pas utilisés, un chercheur qui utilise ce projet peut obtenir davantage de processeurs graphiques. Nous démontrons des scénarios de test sur-quota et des considérations d'équité dans "Une utilisation élevée des clusters grâce à une allocation GPU sur-quota", "Équité de l'allocation des ressources de base", et "Équité excessive".
- Créez un nouveau projet, mettez à jour un projet existant et supprimez un projet existant.
- **Limiter l'exécution des travaux sur des groupes de nœuds spécifiques**. Vous pouvez affecter des projets spécifiques à exécuter uniquement sur des nœuds spécifiques. Ceci est utile lorsque l'équipe de projet a besoin de matériel spécialisé, par exemple, avec suffisamment de mémoire. Une équipe de projet peut également être propriétaire de matériel spécifique acquis avec un budget spécialisé, ou lorsqu'il peut être nécessaire de concevoir directement des charges de travail interactives ou de travailler sur du matériel plus faible et d'entraîner plus longtemps ou des charges de travail non surveillées sur des nœuds plus rapides. Pour des commandes permettant de regrouper des nœuds et de définir l'affinité pour un projet spécifique, reportez-vous à la "Exécuter:Documentation ai".
- **Limiter la durée des emplois interactifs**. Les chercheurs oublient souvent de fermer des emplois interactifs. Cela peut conduire à un gaspillage de ressources. Certaines organisations préfèrent limiter la durée des emplois interactifs et les fermer automatiquement.

La figure suivante montre la vue projets avec quatre équipes créées. Un nombre différent de GPU est attribué à chaque équipe pour prendre en compte différentes charges de travail, avec un nombre total de GPU égal à celui du nombre total de GPU disponibles dans un cluster composé de deux DGX-1.



Soumission des travaux dans l'interface de ligne de commande Run:ai

Cette section fournit des informations détaillées sur les commandes Basic Run:ai que vous pouvez utiliser pour exécuter n'importe quel travail Kubernetes. Elle est divisée en trois parties selon le type de charge de travail. Les workloads d'IA/AM/AP peuvent être divisés en deux types génériques :

- * Sessions de formation sans surveillance*. Lorsque ces types de charges de travail sont évoqués, le data Scientist prépare une charge de travail autonome et l'envoie pour exécution. Lors de l'exécution, le client peut examiner les résultats. Ce type de charge de travail est souvent utilisé en production ou lorsqu'il s'agit du développement du modèle, auquel aucune intervention humaine n'est requise.
- **Sessions de construction interactives**. Avec ce type de charges de travail, un data Scientist ouvre une session interactive avec Bash, Jupyter Notebook, PyChARM distant ou des IDE similaires. Ce dernier accède directement aux ressources GPU. Nous incluons un troisième scénario pour l'exécution de charges de travail interactives avec des ports connectés pour révéler un port interne pour l'utilisateur du conteneur.

Charges de travail d'entraînement sans assistance

Une fois les projets et l'allocation de GPU, vous pouvez exécuter n'importe quel workload Kubernetes à l'aide de la commande suivante sur la ligne de commande :

```
$ runai project set team-a runai submit hyper1 -i gcr.io/run-ai-
demo/quickstart -g 1
```

Cette commande lance une tâche d'entraînement automatique pour TEAM-a avec une allocation d'un GPU unique. La tâche est basée sur un exemple d'image docker, `gcr.io/run-ai-demo/quickstart`. Nous avons nommé le travail `hyper1`. Vous pouvez ensuite contrôler la progression du travail en exécutant la commande suivante :

```
$ runai list
```

La figure suivante montre le résultat du `runai list` commande. Les États typiques que vous pouvez voir incluent les suivants :

- `ContainerCreating`. Le conteneur docker est téléchargé depuis le référentiel cloud.
- `Pending`. Le travail est en attente d'être programmé.
- `Running`. La tâche est en cours d'exécution.

```
~> runai list
Showing jobs for project team-a
NAME    STATUS   AGE  NODE                                    IMAGE                        TYPE         PROJECT  USER   GPUs
hyper1  Running  11s  gke-dev-yaron1-gpu-4-pool-154f511d-5nk5  gcr.io/run-ai-demo/quickstart  Train        team-a   yaron  1
```

Pour obtenir un état supplémentaire de votre travail, exécutez la commande suivante :

```
$ runai get hyper1
```

Pour afficher les journaux du travail, exécutez le `runai logs <job-name>` commande :

```
$ runai logs hyper1
```

Dans cet exemple, vous devriez voir le journal d'une session d'apprentissage profond en cours d'exécution, y

compris la période d'entraînement actuelle, la date d'arrivée prévue, la valeur de la fonction de perte, la précision et le temps écoulé pour chaque étape.

Vous pouvez afficher l'état du cluster dans l'interface utilisateur Run:ai à "https://app.run.ai/". Sous tableaux de bord > Présentation, vous pouvez surveiller l'utilisation des GPU.

Pour arrêter cette charge de travail, exécutez la commande suivante :

```
$ runai delte hyper1
```

Cette commande arrête la charge de travail d'entraînement. Vous pouvez vérifier cette action en exécutant `runai list` encore. Pour plus de détails, voir "lancement de charges de travail d'entraînement non surveillées".

Workloads de construction interactifs

Après avoir configuré des projets et affecté des GPU, vous pouvez exécuter un workload de construction interactif à l'aide de la commande suivante sur la ligne de commande :

```
$ runai submit build1 -i python -g 1 --interactive --command sleep --args
infinity
```

Le travail est basé sur un exemple de python d'image docker. Nous avons nommé le chantier 1.

> Le `-- interactive` drapeau signifie que le travail n'a pas de début ou de fin Il incombe au chercheur de fermer le poste. L'administrateur peut définir une limite de temps pour les travaux interactifs au terme desquels ils sont résiliés par le système.

Le `--g 1` Flag alloue un GPU unique à ce travail. La commande et l'argument fournis sont `--command sleep—args infinity`. Vous devez fournir une commande, ou le conteneur démarre et se ferme immédiatement.

Les commandes suivantes fonctionnent de la même manière que les commandes décrites dans la Charges de travail d'entraînement sans assistance:

- `runai list`: Affiche le nom, l'état, l'âge, le nœud, l'image, Le projet, l'utilisateur et les processeurs graphiques pour la réalisation de tâches.
- `runai get build1`: Affiche l'état supplémentaire sur le build de job 1.
- `runai delete build1`: Arrête la charge de travail interactive build1.pour obtenir un shell bash au conteneur, la commande suivante :

```
$ runai bash build1
```

Ceci fournit un shell direct dans l'ordinateur. Les data Scientists peuvent ensuite développer ou affiner leurs modèles dans le conteneur.

Vous pouvez afficher l'état du cluster dans l'interface utilisateur Run:ai à "https://app.run.ai". Pour plus de

détails, voir "démarrage et utilisation de workloads de construction interactifs".

Charges de travail interactives avec des ports connectés

Comme extension des charges de travail de construction interactives, vous pouvez révéler les ports internes de l'utilisateur du conteneur lors du démarrage d'un conteneur à l'aide de l'interface de ligne de commande Run:ai. Ce procédé est utile pour les environnements cloud, avec des ordinateurs portables Jupyter ou pour la connexion à d'autres microservices. "Entrée" Permet l'accès aux services Kubernetes depuis l'extérieur du cluster Kubernetes. Vous pouvez configurer l'accès en créant un ensemble de règles qui définissent les connexions entrantes qui atteignent quels services.

Pour une meilleure gestion de l'accès externe aux services d'un cluster, nous vous recommandons d'installer les administrateurs du cluster "Entrée" Et configurer LoadBalancer.

Pour utiliser Ingress as a service type, exécutez la commande suivante pour définir le type de méthode et les ports lors de l'envoi de votre workload :

```
$ runai submit test-ingress -i jupyter/base-notebook -g 1 \
  --interactive --service-type=ingress --port 8888 \
  --args="--NotebookApp.base_url=test-ingress" --command=start-notebook.sh
```

Une fois le conteneur démarré avec succès, exécutez `runai list` pour voir le `SERVICE URL(S)` Pour accéder au Jupyter Notebook. L'URL se compose du noeud final d'entrée, du nom du travail et du port. Par exemple, voir https://10.255.174.13/test-ingress-8888.

Pour plus de détails, voir "lancement d'une charge de travail de construction interactive avec des ports connectés".

Optimiser l'utilisation des clusters

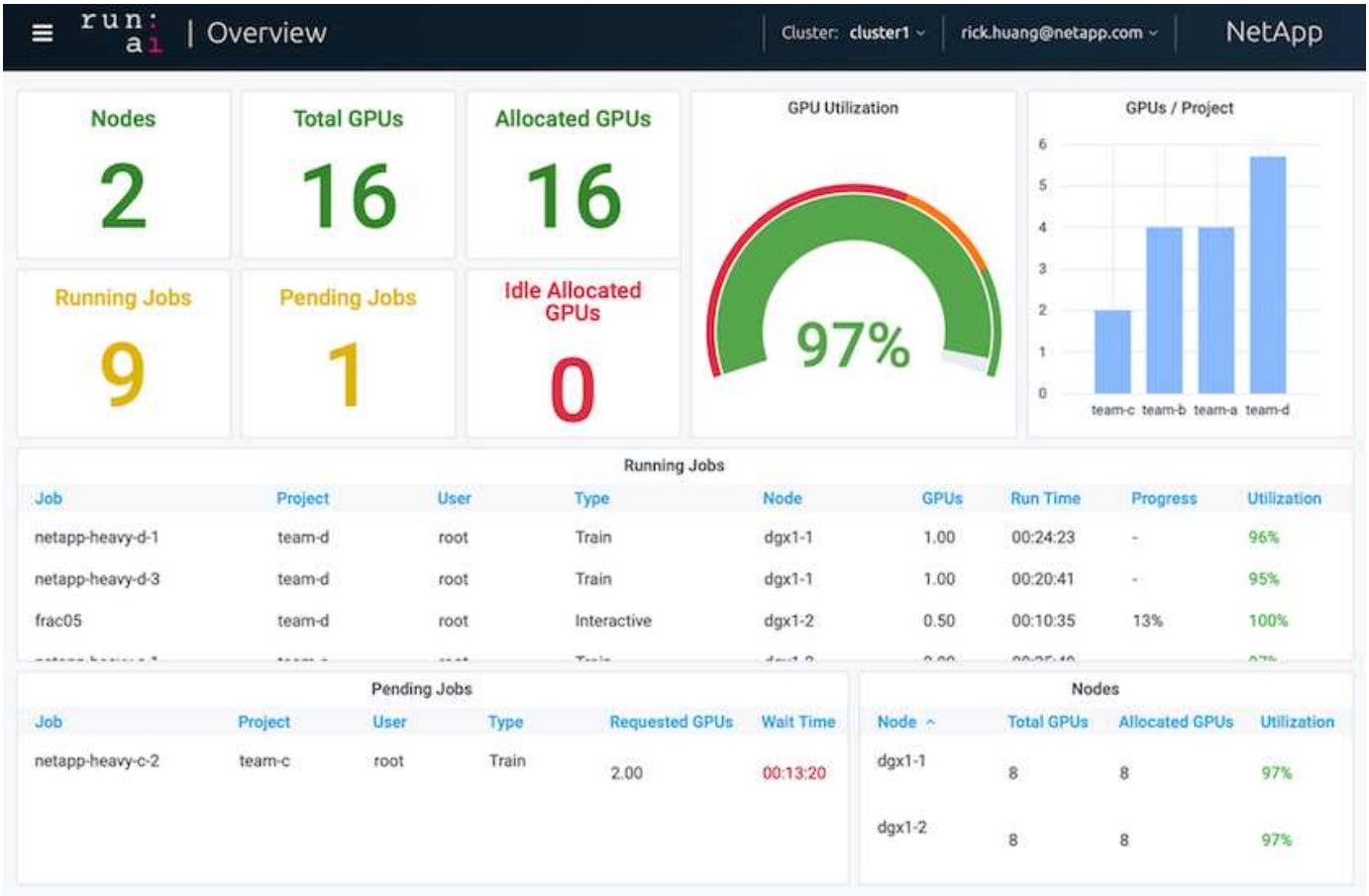
Dans cette section, nous émulons un scénario réaliste dans lequel quatre équipes de data science soumettent leurs propres charges de travail pour présenter la solution d'orchestration Run:ai qui optimise l'utilisation des clusters, tout en maintenant les niveaux de priorité et l'équilibrage des ressources GPU. Nous commençons par le banc d'essai ResNet-50 décrit dans la section "RESNET-50 avec résumé du banc d'essai ImageNet sur le dataset":

```
$ runai submit netapp1 -i netapp/tensorflow-tf1-py3:20.01.0 --local-image
--large-shm  -v /mnt:/mnt -v /tmp:/tmp --command python --args
"/netapp/scripts/run.py" --args "--
dataset_dir=/mnt/mount_0/dataset/imagenet/imagenet_original/" --args "--
num_mounts=2"  --args "--dgx_version=dgx1" --args "--num_devices=1" -g 1
```

Nous avons exécuté le même banc d'essai ResNet-50 que dans "NVA-1121". Nous avons utilisé le drapeau `--local-image` pour les conteneurs qui ne résident pas dans le référentiel docker public. Nous avons monté les répertoires `/mnt` et `/tmp` Sur le nœud hôte DGX-1 vers `/mnt` et `/tmp` vers le conteneur, respectivement. Le dataset est disponible dans le système NetApp AFFA800 avec le `dataset_dir` argument pointant vers le répertoire. Les deux `--num_devices=1` et `-g 1` Cela signifie que nous allouez un GPU pour ce travail. Le

premier est un argument pour le `run.py` script, tandis que ce dernier est un indicateur pour le `runai submit` commande.

La figure ci-dessous montre un tableau de bord de présentation du système avec un taux d'utilisation des GPU de 97 % et les seize GPU disponibles alloués. Vous pouvez consulter le nombre de GPU alloués à chaque équipe dans le graphique à barres projet/GPU. Le volet travaux en cours affiche les noms des travaux en cours d'exécution, le projet, l'utilisateur, le type, le nœud, GPU utilisés, temps d'exécution, progression et informations d'utilisation. Une liste des charges de travail en file d'attente avec leur temps d'attente est affichée dans tâches en attente. Enfin, la zone nœuds fournit les nombres de GPU et l'utilisation de nœuds DGX-1 individuels dans le cluster.



Allocation fractionnelle de GPU pour des charges de travail moins exigeantes ou interactives

Lorsque les chercheurs et développeurs travaillent sur leurs modèles, que ce soit au stade de développement, de réglage des hyperparamètres ou de débogage, ces charges de travail nécessitent généralement moins de ressources de calcul. Il est donc plus efficace de provisionner des GPU et de la mémoire fractionnaires afin que les mêmes GPU puissent être alloués simultanément à d'autres charges de travail. La solution d'orchestration Run:ai propose un système de partage GPU fractionnaire pour les workloads conteneurisés sur Kubernetes. Le système prend en charge les charges de travail exécutant des programmes CUDA et est particulièrement adapté aux tâches d'IA légères telles que l'inférence et la création de modèles. Le système GPU fractionnel permet aux équipes d'ingénierie d'IA et de data science d'exécuter plusieurs charges de

travail simultanément sur un seul GPU. Les entreprises peuvent ainsi exécuter davantage de charges de travail, comme la vision informatique, la reconnaissance vocale et le traitement du langage naturel sur le même matériel, ce qui réduit les coûts.

Run :le système GPU fractionnel de l'IA crée efficacement des GPU logiques virtualisés avec leur propre mémoire et espace de calcul que les conteneurs peuvent utiliser et accéder comme s'il s'agissait de processeurs autonomes. Ce qui permet d'exécuter plusieurs charges de travail côte à côte dans des conteneurs sur le même GPU sans interférer entre les deux. La solution est transparente, simple et portable. Elle ne nécessite aucune modification des conteneurs eux-mêmes.

Une upecase type peut voir deux à huit tâches s'exécutant sur le même GPU, ce qui signifie que vous pouvez faire huit fois le travail avec le même matériel.

Pour le travail `frac05` appartenant au projet `team-d` La figure suivante montre que le nombre de GPU alloués était de 0.50. Ceci est vérifié par le `nvidia-smi` La commande, qui montre que la mémoire GPU disponible pour le conteneur était de 16 255 Mo : la moitié de 32 Go par GPU V100 dans le nœud DGX-1.

```
root@run-deploy:~# runai bash frac05 -p team-d
root@frac05-0:/workload# nvidia-smi
Tue Jul 28 15:17:03 2020
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 450.51.05    Driver Version: 450.51.05    CUDA Version: 11.0     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2...  On   | 00000000:07:00.0 Off |                    0 |
| N/A   57C    P0   240W / 300W |  15525MiB / 16255MiB |     100%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A       156      C   python3                         15525MiB |
+-----------------------------------------------------------------------------+
```

Une utilisation élevée des clusters grâce à une allocation GPU sur-quota

Dans cette section et dans les sections "Équité de l'allocation des ressources de base", et "Équité excessive", Nous avons conçu des scénarios de test avancés pour démontrer les capacités d'orchestration Run:ai pour une gestion de charges de travail complexe, une planification préventive automatique et un provisionnement GPU sur-quota. Nous avons ainsi pu atteindre une utilisation élevée des ressources en cluster et optimiser la

productivité de nos équipes de data Scientists dans un environnement ONTAP d'IA.

Pour ces trois sections, définissez les projets et quotas suivants :

Projet	Quota
équipe a	4
équipe-b	2
equipe-c	2
équipe-d	8

En outre, nous utilisons les conteneurs suivants pour ces trois sections :

- Ordinateur portable Jupyter : `jupyter/base-notebook`
- Run:ai quickstart : `gcr.io/run-ai-demo/quickstart`

Nous avons défini les objectifs suivants dans ce scénario de test :

- Présenter la simplicité du provisionnement des ressources et le mode d'abstraction des ressources des utilisateurs
- Montrer comment les utilisateurs peuvent facilement provisionner des fractions d'un GPU et d'un nombre entier de GPU
- Montrer comment le système élimine les goulots d'étranglement de calcul en permettant aux équipes ou aux utilisateurs de dépasser leur quota de ressources en cas de GPU gratuits dans le cluster
- Montrez comment éliminer les goulots d'étranglement dans les pipelines de données avec la solution NetApp lors de l'exécution de tâches de calcul intensives, telles que le conteneur NetApp
- Montre comment plusieurs types de conteneurs sont exécutés à l'aide du système
  - Ordinateur portable Jupyter
  - Exécutez :conteneur d'IA
- Affiche une utilisation élevée lorsque le cluster est plein

Pour plus de détails sur la séquence de commande réelle exécutée pendant le test, reportez-vous à la section "Détails des tests pour la section 4.8".

Une fois les 13 workloads envoyés, vous voyez la liste des noms de conteneurs et des GPU alloués, comme illustré ci-dessous. Nous avons sept formations et six tâches interactives qui simulent quatre équipes de data science, chacune ayant ses propres modèles en cours de développement ou en cours d'exécution. Pour les travaux interactifs, les développeurs individuels utilisent Jupyter Notebooks pour écrire ou déboguer leur code. Il convient donc de provisionner des fractions GPU sans utiliser trop de ressources du cluster.

```
root@run-deploy:~# runai list -A
NAME           STATUS   AGE  NODE    IMAGE                          TYPE        PROJECT  USER  GPUs  CREATED BY CLI  SERVICE URL(S)
b-4-gg         Running  2m   dgx1-2  gcr.io/run-ai-demo/quickstart  Train       team-b   root  2     true
c-5-g          Running  2m   dgx1-2  gcr.io/run-ai-demo/quickstart  Train       team-c   root  1     true
c-4-gg         Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-c   root  2     true
b-3-g          Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-b   root  1     true
c-3-g02        Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Interactive team-c   root  0.2   true
d-1-gggg       Running  2m   dgx1-2  gcr.io/run-ai-demo/quickstart  Train       team-d   root  4     true
c-2-g03        Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Interactive team-c   root  0.3   true
c-1-g05        Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Interactive team-c   root  0.5   true
a-2-gg         Running  3m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-a   root  2     true
b-2-g04        Running  3m   dgx1-2  gcr.io/run-ai-demo/quickstart  Interactive team-b   root  0.4   true
a-1-g          Running  3m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-a   root  1     true
b-1-g06        Running  3m   dgx1-2  gcr.io/run-ai-demo/quickstart  Interactive team-b   root  0.6   true
a-1-1-jupyter  Running  3m   dgx1-1  jupyter/base-notebook          Interactive team-a   root  1     true            http://10.61.218.134/a-1-1-jupyter,
https://10.61.218.134/a-1-1-jupyter
```

Les résultats de ce scénario de test montrent les éléments suivants :

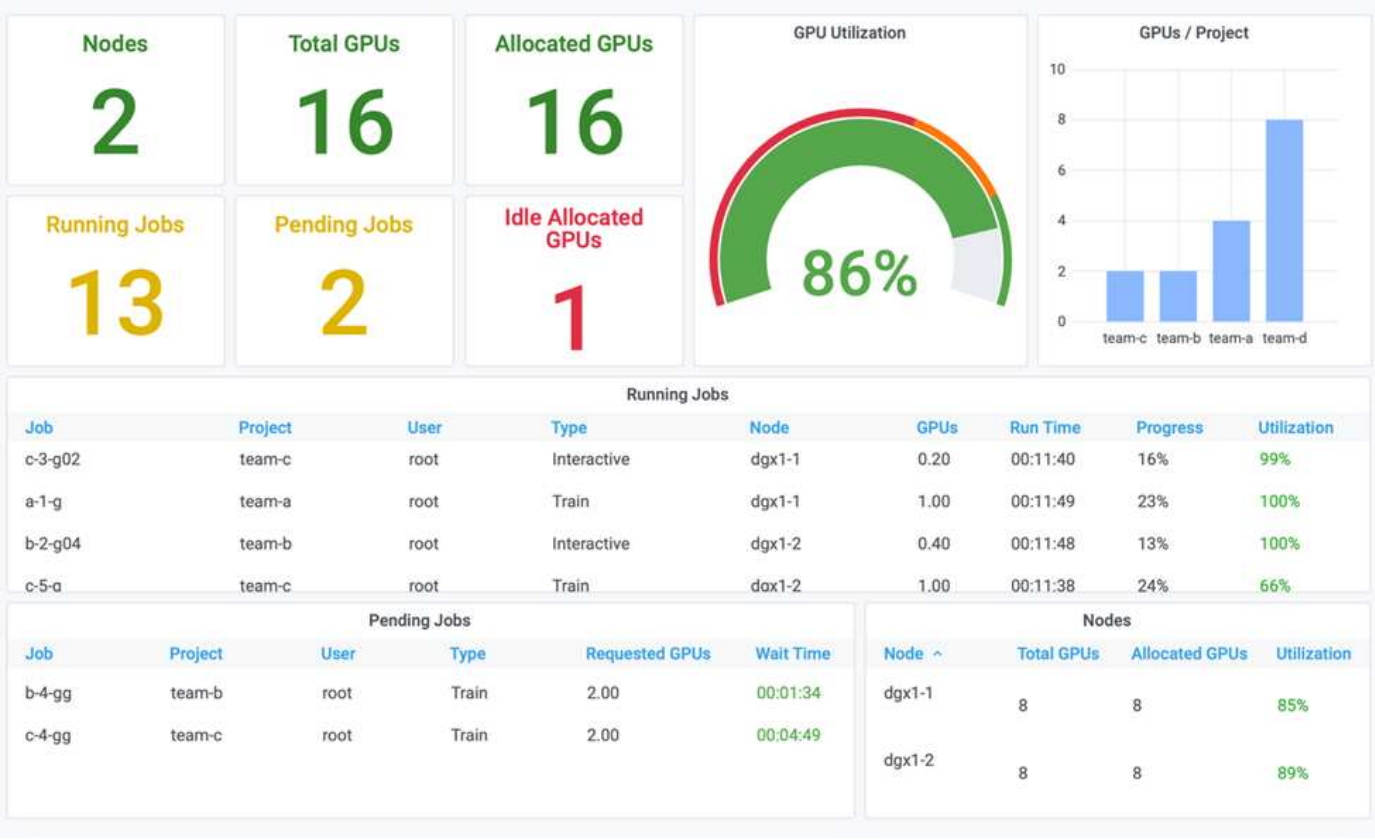
- Le cluster doit être complet : 16/16 GPU sont utilisés.
- Forte utilisation du cluster.
- Plus de tests que les GPU en raison de l'allocation fractionnaire.
- `team-d` n'utilise pas tous leurs quotas ; par conséquent, `team-b` et `team-c` Peuvent exploiter des GPU supplémentaires pour leurs expérimentations, ce qui permet d'accélérer l'innovation.

Équité de l'allocation des ressources de base

Dans cette section, quand nous le montrons `team-d` Lorsque plusieurs GPU sont en attente (ils sont sous leur quota), le système interrompt les charges de travail de `team-b` et `team-c` et les déplace dans un état en attente de façon équitable.

Pour plus de détails, y compris les soumissions de travaux, les images de conteneur utilisées et les séquences de commandes exécutées, reportez-vous à la section "Détails des tests pour la section 4.9".

La figure suivante montre l'utilisation du cluster résultant, les GPU alloués par équipe et les tâches en attente dus à l'équilibrage de la charge automatique et à la planification préventive. Nous pouvons observer que lorsque le nombre total de GPU requis par toutes les charges de travail d'équipe dépasse le nombre total de GPU disponibles dans le cluster, l'algorithme d'équité interne de Run:ai interrompt chaque tâche pour `team-b` et `team-c` parce qu'ils ont atteint leur quota de projet. Cela assure, de manière générale, l'utilisation élevée des clusters et les équipes de data Scientists restent contraintes pour les ressources définies par un administrateur.



Les résultats de ce scénario de test démontrent ce qui suit :

- **Équilibrage automatique de la charge.** le système équilibre automatiquement le quota des GPU, de sorte que chaque équipe utilise maintenant son quota. Les charges de travail suspendues appartiennent à des équipes qui ont terminé leur quota.
- **Pause de partage équitable.** le système choisit d'arrêter la charge de travail d'une équipe qui a surpassé leur quota et de mettre fin à la charge de travail de l'autre équipe. Exécutez :l'IA dispose d'algorithmes d'équité internes.

Équité excessive

Dans cette section, nous étudions le scénario dans lequel plusieurs équipes soumettent des charges de travail pour dépasser leurs quotas. De cette façon, nous démontrons comment l'algorithme Run:ai d'équité alloue des ressources de cluster en fonction du rapport des quotas prédéfinis.

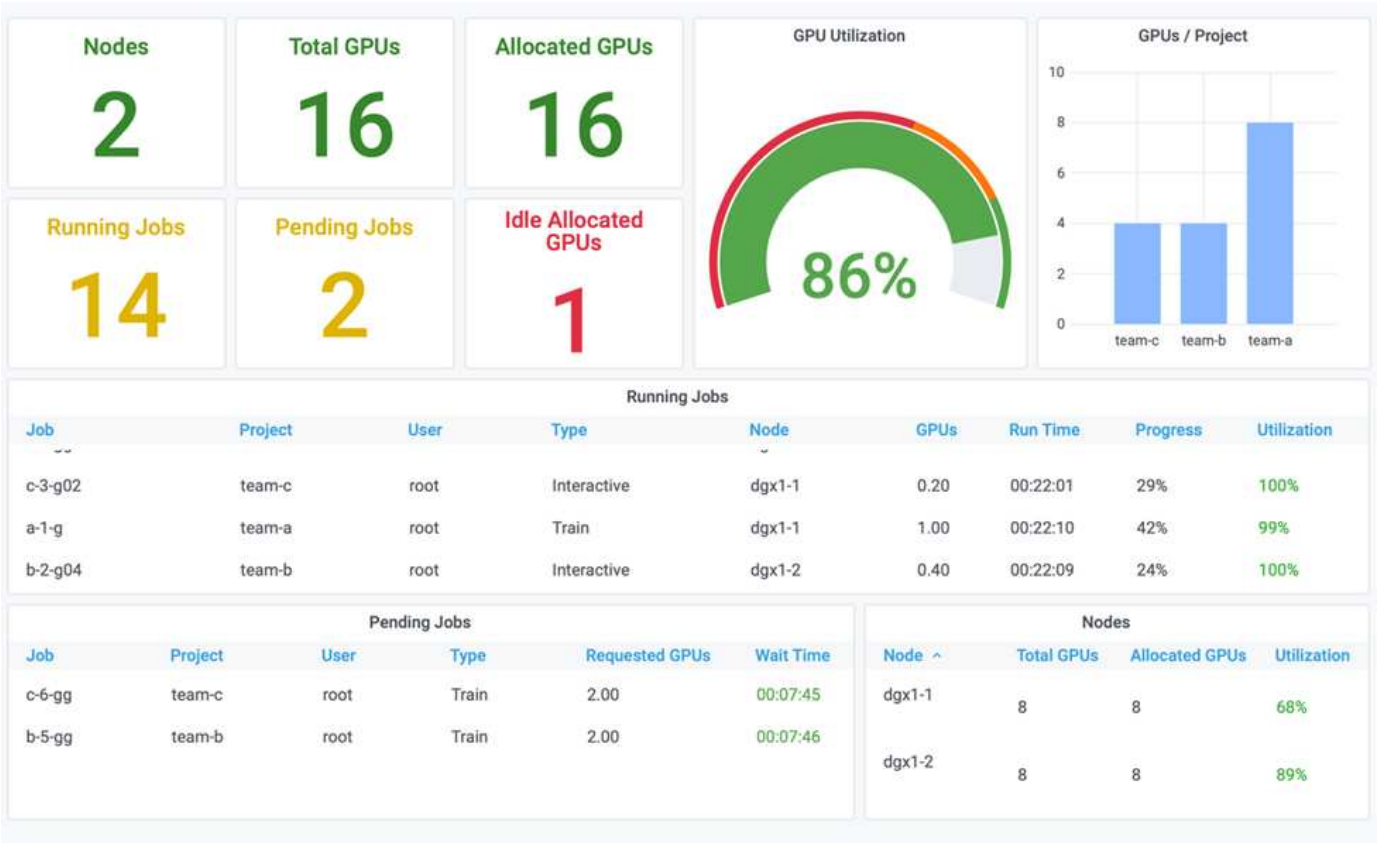
Objectifs de ce scénario de test :

- Montrer le mécanisme de mise en file d'attente lorsque plusieurs équipes demandent des GPU sur leur quota.
- Montrer comment le système distribue une juste part du cluster entre plusieurs équipes qui surpartagent leur quota d'après le rapport entre leurs quotas, de sorte que l'équipe disposant du quota supérieur puisse obtenir une plus grande part de la capacité disponible.

À la fin de "Équité de l'allocation des ressources de base", deux charges de travail sont mises en file d'attente : une pour `team-b` et un pour `team-c`. Cette section décrit les workloads supplémentaires mis en file d'attente.

Pour plus de détails, y compris les soumissions de travaux, les images de conteneur utilisées et les séquences de commandes exécutées, voir "Détails des tests pour la section 4.10".

Lorsque tous les travaux sont soumis conformément à la section "Détails des tests pour la section 4.10", le tableau de bord du système le montre `team-a`, `team-b`, et `team-c` Tous ont plus de GPU que leur quota prédéfini. `team-a` Occupe quatre GPU de plus que son soft quota (quatre) prédéfini, alors que `team-b` et `team-c` Chacun d'eux occupent deux GPU de plus que leur soft quota (deux). Le rapport des GPU sur-quotas alloués est égal à celui de leur quota prédéfini. En effet, le système a utilisé le quota prédéfini comme référence en priorité et provisionné en conséquence lorsque plusieurs équipes demandent plus de GPU, ce qui dépasse leur quota. Un tel équilibrage automatique de la charge garantit équité et priorisation lorsque les équipes de data science d'entreprise sont activement engagées dans le développement et la production de modèles d'IA.



Les résultats de ce scénario de test montrent les éléments suivants :

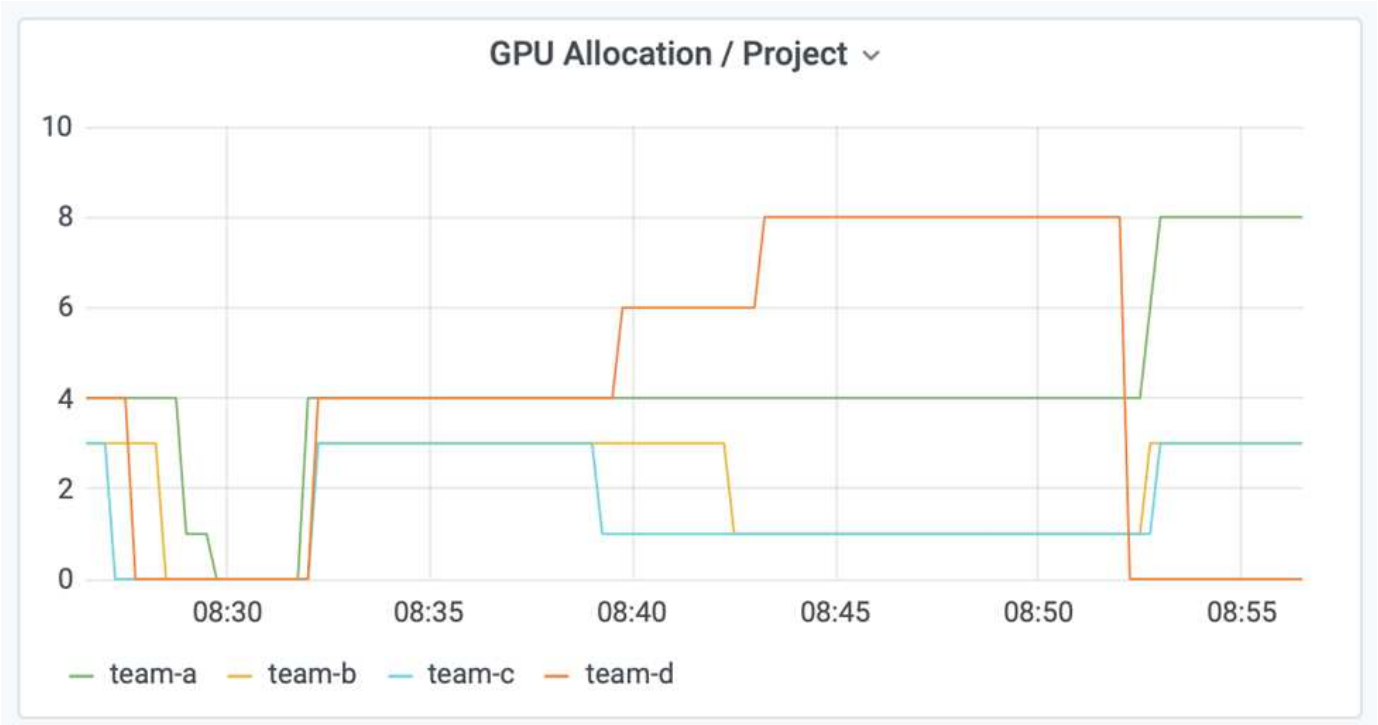
- Le système commence à mettre hors file d'attente les charges de travail des autres équipes.
- L'ordre de la démise en file d'attente est déterminé selon des algorithmes d'équité, de telle sorte que `team-b` et `team-c` Obtenir la même quantité de GPU sur-quota (puisqu'ils ont un quota similaire), et `team-a` Double quantité de GPU dans la mesure où le quota est deux fois plus élevé que celui de `team-b` et `team-c`.
- Toute l'allocation est effectuée automatiquement.

Par conséquent, le système doit se stabiliser sur les États suivants :

Projet	GPU alloués	Commentaire
équipe a	8/4	Quatre GPU par rapport au quota. File d'attente vide.

Projet	GPU alloués	Commentaire
équipe-b	4/2	Deux GPU sur le quota. Une charge de travail mise en file d'attente
equipe-c	4/2	Deux GPU sur le quota. Une charge de travail mise en file d'attente
équipe-d	0/8	N'utilisez pas du tout des GPU, aucun workload mis en attente.

La figure suivante montre l'allocation des GPU par projet dans le temps dans le tableau de bord Run:ai Analytics "Une utilisation élevée des clusters grâce à une allocation GPU sur-quota", "Équité de l'allocation des ressources de base", et "Équité excessive". Chaque ligne de la figure indique le nombre de GPU provisionnés pour une équipe de data science donnée à tout moment. Le système obtient ainsi l'allocation dynamique de GPU en fonction des charges de travail envoyées. Cette approche permet aux équipes de dépasser les quotas en présence de GPU disponibles dans le cluster, puis d'anticiper les tâches par équité avant d'atteindre la stabilité des quatre équipes.



Sauvegarde des données vers un volume persistant provisionné par Trident

NetApp Trident est un projet open source entièrement pris en charge et conçu pour vous aider à répondre aux exigences de persistance sophistiquées de vos applications conteneurisées. Vous pouvez lire et écrire des données sur un volume persistant Kubernetes persistant provisionné par Trident avec le Tiering des données, le chiffrement, la technologie NetApp Snapshot, la conformité et les performances élevées proposés par le logiciel de gestion des données NetApp ONTAP.

Réutilisation des ESV dans un espace de noms existant

Pour les projets d'IA plus volumineux, il est possible que les conteneurs soient plus efficaces pour lire et écrire les données sur le même volume persistant Kubernetes. Pour réutiliser une demande de volume persistant Kubernetes, l'utilisateur doit avoir déjà créé une demande de volume persistant. Voir la "Documentation NetApp Trident" Pour plus de détails sur la création d'une demande de volume persistant. Voici un exemple de réutilisation d'un PVC existant :

```
$ runai submit pvc-test -p team-a --pvc test:/tmp/pvc1mount -i gcr.io/run-
ai-demo/quickstart -g 1
```

Exécutez la commande suivante pour afficher le statut de cette tâche `pvc-test` pour projet `team-a`:

```
$ runai get pvc-test -p team-a
```

Vous devriez voir le PV /tmp/pvc1mount monté sur `team-a` travail `pvc-test`. Ainsi, plusieurs conteneurs peuvent être lus depuis le même volume, ce qui est utile lorsque plusieurs modèles concurrents sont en cours de développement ou en production. Les data Scientists peuvent élaborer un ensemble de modèles, puis combiner les résultats de prédiction par vote à la majorité ou d'autres techniques.

Pour accéder au shell du conteneur, procédez comme suit :

```
$ runai bash pvc-test -p team-a
```

Vous pouvez ensuite vérifier le volume monté et accéder à vos données dans le conteneur.

Cette fonctionnalité de réutilisation des volumes virtuels fonctionne avec des volumes NetApp FlexVol et des volumes NetApp ONTAP FlexGroup. Les ingénieurs de données peuvent ainsi utiliser des options de gestion des données plus flexibles et plus robustes pour exploiter votre Data Fabric optimisée par NetApp.

Informations sur le copyright

Copyright © 2024 NetApp, Inc. Tous droits réservés. Imprimé aux États-Unis. Aucune partie de ce document protégé par copyright ne peut être reproduite sous quelque forme que ce soit ou selon quelque méthode que ce soit (graphique, électronique ou mécanique, notamment par photocopie, enregistrement ou stockage dans un système de récupération électronique) sans l'autorisation écrite préalable du détenteur du droit de copyright.

Les logiciels dérivés des éléments NetApp protégés par copyright sont soumis à la licence et à l'avis de non-responsabilité suivants :

CE LOGICIEL EST FOURNI PAR NETAPP « EN L'ÉTAT » ET SANS GARANTIES EXPRESSES OU TACITES, Y COMPRIS LES GARANTIES TACITES DE QUALITÉ MARCHANDE ET D'ADÉQUATION À UN USAGE PARTICULIER, QUI SONT EXCLUES PAR LES PRÉSENTES. EN AUCUN CAS NETAPP NE SERA TENU POUR RESPONSABLE DE DOMMAGES DIRECTS, INDIRECTS, ACCESSOIRES, PARTICULIERS OU EXEMPLAIRES (Y COMPRIS L'ACHAT DE BIENS ET DE SERVICES DE SUBSTITUTION, LA PERTE DE JOUISSANCE, DE DONNÉES OU DE PROFITS, OU L'INTERRUPTION D'ACTIVITÉ), QUELLES QU'EN SOIENT LA CAUSE ET LA DOCTRINE DE RESPONSABILITÉ, QU'IL S'AGISSE DE RESPONSABILITÉ CONTRACTUELLE, STRICTE OU DÉLICTUELLE (Y COMPRIS LA NÉGLIGENCE OU AUTRE) DÉCOULANT DE L'UTILISATION DE CE LOGICIEL, MÊME SI LA SOCIÉTÉ A ÉTÉ INFORMÉE DE LA POSSIBILITÉ DE TELS DOMMAGES.

NetApp se réserve le droit de modifier les produits décrits dans le présent document à tout moment et sans préavis. NetApp décline toute responsabilité découlant de l'utilisation des produits décrits dans le présent document, sauf accord explicite écrit de NetApp. L'utilisation ou l'achat de ce produit ne concède pas de licence dans le cadre de droits de brevet, de droits de marque commerciale ou de tout autre droit de propriété intellectuelle de NetApp.

Le produit décrit dans ce manuel peut être protégé par un ou plusieurs brevets américains, étrangers ou par une demande en attente.

LÉGENDE DE RESTRICTION DES DROITS : L'utilisation, la duplication ou la divulgation par le gouvernement sont sujettes aux restrictions énoncées dans le sous-paragraphe (b)(3) de la clause Rights in Technical Data-Noncommercial Items du DFARS 252.227-7013 (février 2014) et du FAR 52.227-19 (décembre 2007).

Les données contenues dans les présentes se rapportent à un produit et/ou service commercial (tel que défini par la clause FAR 2.101). Il s'agit de données propriétaires de NetApp, Inc. Toutes les données techniques et tous les logiciels fournis par NetApp en vertu du présent Accord sont à caractère commercial et ont été exclusivement développés à l'aide de fonds privés. Le gouvernement des États-Unis dispose d'une licence limitée irrévocable, non exclusive, non cessible, non transférable et mondiale. Cette licence lui permet d'utiliser uniquement les données relatives au contrat du gouvernement des États-Unis d'après lequel les données lui ont été fournies ou celles qui sont nécessaires à son exécution. Sauf dispositions contraires énoncées dans les présentes, l'utilisation, la divulgation, la reproduction, la modification, l'exécution, l'affichage des données sont interdits sans avoir obtenu le consentement écrit préalable de NetApp, Inc. Les droits de licences du Département de la Défense du gouvernement des États-Unis se limitent aux droits identifiés par la clause 252.227-7015(b) du DFARS (février 2014).

Informations sur les marques commerciales

NETAPP, le logo NETAPP et les marques citées sur le site http://www.netapp.com/TM sont des marques déposées ou des marques commerciales de NetApp, Inc. Les autres noms de marques et de produits sont des marques commerciales de leurs propriétaires respectifs.